

Smaatech HRMS

Backend API Specification

Prepared for: Backend Developer

Version 1.0 - 2026-07-03

Confidential - Internal Handoff Document

Table of Contents

Table of Contents	2
1. Executive Overview	5
2. Tech Stack and Migration Strategy	6
2.1 Current frontend stack	6
2.2 Suggested backend approach	6
2.3 Migration approach	6
2.4 Minimal Express app skeleton	6
3. Authentication and Authorization Design	8
3.1 Endpoints	8
3.2 Session strategy - pick one	8
3.3 Role to permission mapping	8
3.4 Migrating the existing 4 demo accounts	9
3.5 Face-recognition login	9
3.6 Express example - login route	9
4. Core REST Conventions	10
4.1 Base path	10
4.2 The generic collection pattern	10
4.3 Pagination	10
4.4 Error envelope	10
4.5 IDs and timestamps	10
5. Per-Module Endpoint Reference	11

5.1 Employees	11
5.2 Leave	12
5.3 Attendance	12
5.4 Payroll	13
5.5 Celebrations, Holidays, Jobs, Assets	14
5.6 Recruitment	14
5.7 Performance Reviews	15
5.8 Expenses	16
5.9 Settings	16
6. File Upload and Storage Design	18
6.1 Endpoints	18
6.2 Storage	18
6.3 Size limits and allowed types	18
6.4 Visibility	18
6.5 Express example	18
7. Simulated Features That Need Real Implementation	20
8. Non-Blocking and Future Items	21
Appendix: Full JSON Field Reference	22
Employee	22
Leave	22
Attendance	22
Payroll	22
Celebration	22
Holiday	22

Candidate (Recruitment)	22
Job Posting	23
Performance Review	23
Expense	23
Asset	23
Settings (singleton, auth fields removed - see Section 3)	23

1. Executive Overview

Smaatech HRMS is currently a fully client-side React application. There is no backend server today: every screen reads and writes through a single mock data-access module (`src/data/store.js`) that persists everything to the browser's `localStorage` and returns Promises, so the rest of the app already treats data access as asynchronous. The code was written with a real backend in mind from day one - the store's own comments say so directly.

Core contract: Every collection in `store.js` (`employeesApi`, `payrollApi`, etc.) exposes exactly four methods: `list` / `get` / `create` / `update` / `remove`. Each one maps to a single REST verb. If your API returns the same JSON shapes the frontend already expects, no frontend component code needs to change - only the five functions inside `store.js` get rewritten to call `fetch()` instead of `localStorage`.

What already exists and works end-to-end today (frontend only)

- 19 fully-built pages: Dashboard, My Dashboard (employee self-service), Employees, Org Chart, Attendance, Leave, Holidays, Payroll, Celebrations, Recruitment, Performance, Documents, Analytics, Settings, Integrations, Expenses, Assets, Workflows, and Login.
- Real CRUD, search, filter, sort, pagination, forms with manual validation, modals and confirm dialogs, role-based navigation and route guarding for 4 roles.
- A single React Context (`HRMSContext.jsx`) that every page reads and writes through - this is effectively the 'client SDK' your API needs to support.

What is simulated today and needs real backend (and sometimes frontend) work

- Login / sessions - 4 hardcoded plaintext email+password pairs, no server verification.
- OTP / two-factor - a fake code is shown to the user in a toast notification.
- QR check-in - a cosmetic QR graphic that auto-succeeds on a timer, nothing is decoded.
- Biometric device sync - fully randomized, no real device protocol is spoken.
- SMS / Email / WhatsApp sending - templates and credential fields exist, nothing is ever sent.
- Resume upload (Recruitment) and expense receipts - hardcoded text / pasted URLs, not real files.
- GPS geofence check-in - validated only in the browser, trivially bypassable today.

Section 7 covers every one of these in a single reference table with exactly what needs to change.

2. Tech Stack and Migration Strategy

2.1 Current frontend stack

- React 18.3 with plain JavaScript (JSX) - no TypeScript.
- Build tool: Vite 5. Routing: react-router-dom v6.
- State: one global React Context, no Redux/Zustand.
- Styling: a single hand-written stylesheet, no UI component library.
- face-api.js for client-side face-recognition login (runs entirely in the browser).
- No axios, no fetch calls, no environment variables configured yet - a clean slate for wiring up a real API base URL (e.g. VITE_API_BASE_URL).

2.2 Suggested backend approach

This document's examples use **Node.js + Express**, matching the request for this handoff. The choice of database is left to you; given how relational the data is (employees self-reference a manager, and leaves/attendance/payroll/reviews/expenses all foreign-key into employees), a relational database such as PostgreSQL or MySQL is a natural fit, though a document store like MongoDB will also work since most records are simple flat objects. The Express examples below use Mongoose-style pseudocode (`Model.find()`, `Model.create()`) purely as illustrative shorthand - swap in your ORM/query builder of choice.

2.3 Migration approach

Replace the five functions returned by `createResource()` in `src/data/store.js` one collection at a time. Keep the exact same async/Promise signature so nothing else in the app has to change:

```
// BEFORE (src/data/store.js) - localStorage version
list: () => wait(clone(db[collection])),

// AFTER - real backend version, same call signature
list: () => fetch(`${API_BASE}/${collection}`).then(r => r.json()),
```

2.4 Minimal Express app skeleton

```
// server.js
const express = require('express');
const cors = require('cors');
const app = express();

app.use(cors());
app.use(express.json());

app.use('/api/v1/auth', require('./routes/auth'));
app.use('/api/v1/employees', require('./routes/employees'));
app.use('/api/v1/leaves', require('./routes/leaves'));
app.use('/api/v1/attendance', require('./routes/attendance'));
app.use('/api/v1/payroll', require('./routes/payroll'));
app.use('/api/v1/recruitment', require('./routes/recruitment'));
app.use('/api/v1/reviews', require('./routes/reviews'));
app.use('/api/v1/expenses', require('./routes/expenses'));
app.use('/api/v1/assets', require('./routes/assets'));
app.use('/api/v1/settings', require('./routes/settings'));
app.use('/api/v1/files', require('./routes/files'));

app.listen(process.env.PORT || 4000);
```

3. Authentication and Authorization Design

Today's login has no server component at all: 4 demo profiles with plaintext passwords live in `src/lib/permissions.js`, OTP is a fake code shown in a toast, 'forgot password' resets the password directly in browser state, and the logged-in session is just an object written to `localStorage` that is never re-checked against anything. All of this needs a real implementation.

3.1 Endpoints

Method	Path	Description
POST	/api/v1/auth/login	Body: email, password. Returns a session/token, or requiresOtp: true
POST	/api/v1/auth/otp/verify	Body: email, code. Completes login after 2FA
POST	/api/v1/auth/otp/resend	Body: email. Re-sends the one-time code
POST	/api/v1/auth/logout	Invalidates the current session/refresh token
GET	/api/v1/auth/me	Returns the authenticated user's profile and role
POST	/api/v1/auth/forgot-password	Body: email. Sends a real reset link/email
POST	/api/v1/auth/reset-password	Body: token, newPassword

3.2 Session strategy - pick one

Option A - JWT: short-lived access token (about 15 minutes) kept in memory on the frontend, plus an `httpOnly` refresh cookie (7-30 days). Scales cleanly to a future mobile app or additional services, and avoids ever putting a long-lived token where client-side JavaScript can read it.

Option B - Server session cookie: simpler to implement and to revoke immediately (just delete the session row), fine for this app's scale. No token-replay surface to think about.

Either is acceptable for this project; JWT is the slightly more future-proof default if you have no strong preference.

3.3 Role to permission mapping

Today's frontend only checks *which page* a role can open (`canAccess()` in `src/lib/permissions.js`). A finer-grained action table (`canDo()`) exists in the same file but is never actually called anywhere in the app - so the backend cannot inherit real per-action authorization from the frontend. Use the table below (assembled from what each page actually does) as the source of truth for server-side authorization instead:

Role	Access
HR Director	Full access to every endpoint (superuser).
HR Manager	Employees, Attendance, Leave, Holidays, Celebrations, Recruitment, Performance, Assets, Workflows - full read/write. Expenses - read only. No Payroll, no Settings write.
Finance Lead	Payroll, Documents, Expenses (approve/decline), Assets, Analytics - full read/write. No Employee record write, no Leave approval.
Employee	Own records only: own leave requests, own attendance check-in/out, own payroll (read-only), own expense claims, own performance self-review.

3.4 Migrating the existing 4 demo accounts

Seed a real users table with the same 4 identities so the demo experience keeps working end-to-end after go-live: an HR Director, an HR Manager, a Finance Lead, and an Employee account linked to an existing employee record. **Hash every password with bcrypt or argon2 before storing it** - never carry the plaintext passwords from `permissions.js` into the new database or into any shared document; ask whoever owns the frontend for the current demo credentials directly.

3.5 Face-recognition login

Face matching runs entirely client-side today via `face-api.js`, comparing a live webcam capture against a 128-number face descriptor stored per profile in the browser. This can stay fully client-side. If you want server-side verification instead, add a `user_face_descriptors` table (`userId`, `descriptor` as a JSON float array) and an endpoint to fetch/store it - treat this as optional, not a launch blocker.

3.6 Express example - login route

```
// routes/auth.js
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');

router.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (!user || !(await bcrypt.compare(password, user.passwordHash))) {
    return res.status(401).json({ error: { code: 'INVALID_CREDENTIALS', message: 'Wrong email or password' } });
  }
  if (user.twoFactorEnabled) {
    await sendOtp(user.email); // real SMS/email provider, see Section 7
    return res.json({ requiresOtp: true });
  }
  const token = jwt.sign({ sub: user.id, role: user.role }, process.env.JWT_SECRET, { expiresIn: '15m' });
  res.json({ token, user: { id: user.id, name: user.name, role: user.role } });
});
```

4. Core REST Conventions

4.1 Base path

All endpoints in this document are relative to `/api/v1`.

4.2 The generic collection pattern

Every simple collection (celebrations, holidays, jobs, assets, and the base shape of employees/leaves/attendance/payroll/recruitment/reviews/expenses) follows the same five-route pattern, matching the five methods already used throughout the frontend's data layer:

Method	Path	Description
GET	/collection	List records (supports ?page= and ?pageSize=)
GET	/collection/:id	Fetch a single record by id
POST	/collection	Create a new record
PATCH	/collection/:id	Partially update a record
DELETE	/collection/:id	Remove a record

4.3 Pagination

The frontend currently paginates client-side (e.g. the Employees list slices 6 records per page in the browser), so it can safely ignore pagination metadata it doesn't yet use. Still, build every list endpoint to accept ?page=&pageSize= and return an envelope so the frontend can move to server-side pagination later without an API version bump:

```
{ "data": [ /* records */ ], "page": 1, "pageSize": 20, "total": 137 }
```

4.4 Error envelope

Use a consistent error shape on every 4xx/5xx response so the frontend can show one generic error handler everywhere:

```
{ "error": { "code": "NOT_FOUND", "message": "Employee not found" } }
```

4.5 IDs and timestamps

The frontend does not care what format ids are in (today's mock layer generates strings like emp_a1b2c3) - UUIDs or auto-increment integers-as-strings are both fine. Recommend adding createdAt / updatedAt to every table even though the current frontend doesn't display them yet - useful for the audit log feature in Settings, and cheap to add now versus retrofitting later.

5. Per-Module Endpoint Reference

Every module below follows the same shape: Endpoints, then the request/response JSON, who's allowed to call it, and any business logic the frontend currently does client-side that the backend must now own. Modules that add nothing beyond the generic pattern in Section 4 are noted as such rather than repeated in full.

5.1 Employees

Method	Path	Description
GET	/employees	List all employees
GET	/employees/:id	Get one employee
POST	/employees	Create an employee
PATCH	/employees/:id	Update an employee
DELETE	/employees/:id	Remove an employee
POST	/employees/import	Bulk-create from a validated CSV row array

Record shape

```
{
  "id": "emp_a1b2c3", "name": "Ananya Nair", "role": "Senior SDE", "dept": "Engineering",
  "loc": "Bengaluru", "email": "ananya.nair@smaatech.co", "phone": "+91 98000 00001",
  "status": "active",           // active | remote | on-leave
  "joinDate": "2019-03-11", "salary": 236666, "rating": 4.9,
  "managerId": "emp_x9y8z7", "bankAccount": "", "ifsc": ""
}
```

Roles

Read: HR Director, HR Manager, Finance Lead (list/read only). Own record: Employee, via /ess. Create / update / delete: HR Director and HR Manager only.

Business logic to own

- On create: also create today's attendance row (absent, or leave if status is on-leave) and a payroll row for the current cycle (deductions = 30% of gross as a placeholder - confirm the real statutory calculation with Finance before treating this as final).
- On update: if name or department changes, cascade the change onto that employee's denormalized name/dept copies in attendance, payroll and leave rows.
- On update: if salary changes, decide (with the product owner) whether to recompute future payroll cycles only, or also rewrite already-paid historical cycles - today's frontend rewrites everything including paid cycles, which is likely not the desired real-world behavior.
- On delete: cascade-delete that employee's attendance/payroll/leave/review rows, null out managerId on any direct reports, and unlink any login account pointed at them.

- /employees/import: validate every row server-side (required name/role, dept must exist in settings.departments, valid email, non-negative numeric salary, optional managerName resolved to a managerId) and return a per-row success/failure result - do not just loop individual creates blindly.

5.2 Leave

Method	Path	Description
GET	/leaves	List leave requests
POST	/leaves	Create a leave request
PATCH	/leaves/:id	Update status (approve/decline) or edit
DELETE	/leaves/:id	Remove a leave request

Record shape

```
{
  "id": "lv_9f8e7d", "empId": "emp_a1b2c3", "name": "Priya Sharma", "dept": "Engineering",
  "type": "sick",           // sick | earned | casual
  "start": "2026-06-02", "end": "2026-06-04",
  "status": "pending",     // pending | approved | declined
  "reason": "Fever and rest advised by doctor."
}
```

Roles

Create: Employee (self) or HR Manager/Director on anyone's behalf. Approve/decline/delete: HR Manager, HR Director only.

Business logic to own

- When status changes to approved and today falls within start/end: set that employee's status to on-leave and set today's attendance row to status leave with checkIn/checkOut cleared. No side effect on decline.
- Multi-step approval chains are not implemented anywhere in the current frontend - see Section 8 (Workflows is a saved-but-unused setting today). Treat as optional/v2.

5.3 Attendance

Method	Path	Description
GET	/attendance	List today's (or a date range of) attendance rows
PATCH	/attendance/:id	Manual status override by HR
POST	/attendance/:id/check-in	Employee self check-in
POST	/attendance/:id/check-out	Employee self check-out
POST	/attendance/qr-token	HR-only: generate a short-lived signed QR token

Record shape

```
{
  "id": "att_112233", "empId": "emp_a1b2c3", "name": "Ananya Nair", "dept": "Engineering",
  "date": "2026-07-03", "checkIn": "09:14", "checkOut": null,
  "status": "present"           // present | late | absent | leave
}
```

Roles

HR Manager/Director: full read and override for anyone. Employee: check-in/out for self only.

Business logic to own - this is the module with the most client-side trust to remove

- Late threshold: checkIn time after 09:30 marks status late, otherwise present - make this a configurable setting rather than hardcoded, and compute it server-side from server time.
- GPS geofencing: when settings.gpsCheckInEnabled is on, the frontend already computes a Haversine distance between the device's coordinates and settings.geofenceLat/Lng, and refuses check-in beyond geofenceRadius meters - but only in the browser. A modified client can currently submit a check-in with no coordinates at all. The server must re-run this same distance check and reject the request, never trust a client-supplied 'inside geofence' flag.
- QR check-in needs a real design, since today it auto-succeeds after a 3-second timer with no payload check: have the office screen call POST /attendance/qr-token to display a short-lived signed token (10-30 second expiry is plenty), have the employee's device read it, then send it back as { qrToken } on the check-in call - reject if the signature or expiry doesn't check out.
- Biometric device punches: keep a reconciliation endpoint, e.g. POST /attendance/punches/reconcile { emplId, time, type }, for the staging queue UI in Integrations - but actually polling/ingesting from real ZKTeco/eSSL hardware is a separate, larger integration project (today it is 100% randomly generated) - see Section 8.

5.4 Payroll

Method	Path	Description
GET	/payroll	List payroll rows (optionally ?cycle=2026-07)
PATCH	/payroll/:id	Mark a single row paid, or edit
POST	/payroll/:id/structure	Save the earnings/deductions breakdown for one employee
POST	/payroll/process	Bulk-process all ready rows for a given cycle

Record shape

```
{
  "id": "pay_556677", "empId": "emp_a1b2c3", "name": "Ananya Nair", "dept": "Engineering",
  "gross": 236666, "deductions": 71000, "net": 165666,
  "status": "ready", // ready | processing | paid
  "cycle": "2026-07",
  "lopDays": 0, "lopAmount": 0,
  "components": { "earnings": [], "deductions": [] }
}
```

Roles

Finance Lead, HR Director only. Employees see only their own rows (read-only), e.g. via GET /payroll?empld=me on the ESS dashboard.

Business logic to own

- Structure endpoint computes: gross = sum(earnings), deductions = sum(deduction components), lopAmount = round(gross / 30 * lopDays), net = gross - deductions - lopAmount.
- POST /payroll/process should be scoped to a single { cycle } - today's frontend marks every ready row across every cycle as paid in one click, which is very likely not the intended real-world scope; confirm with the product owner and scope it correctly server-side regardless.
- Payslip 'download' today is really an HTML file renamed with a .html extension, not a real PDF. Decide whether the backend should generate genuine PDF payslips (recommended for a production system) or the frontend keeps doing print-to-PDF in the browser.
- The bank-advice CSV export is generated fully client-side from payroll + employee bank details already in memory - no new endpoint needed, it just needs to be fed real data.

5.5 Celebrations, Holidays, Jobs, Assets

These four follow the generic collection pattern from Section 4.2 with a couple of small action endpoints layered on top - no dedicated Express example is included here since the generic CRUD router in Section 2.4 covers the bulk of it.

Module	Fields	Notes
Celebrations	id, type (birthday/anniv), name, detail, wished (bool)	POST /celebrations/:id/wish sets wished true and increments a wish counter - no real notification is sent, this is a cosmetic 'send a wish' action today.
Holidays	id, name, date, type (National/Regional/Optional)	Plain CRUD, HR Manager/Director write access, everyone else read-only.
Jobs (postings)	id, title, department, location, type, status (Open/Closed), description	PATCH /jobs/:id toggles Open/Closed. HR Manager/Director write access.
Assets	id, name, category, serialNumber, status (assigned/available), assignedToEmpId, assignedToEmpName, assignedDate	POST /assets/:id/assign { empld, date } and POST /assets/:id/return as small action endpoints on top of the generic pattern.

5.6 Recruitment

Method	Path	Description
GET	/candidates	List candidates across the pipeline
POST	/candidates	Add a candidate to a stage
PATCH	/candidates/:id	Move stage, edit fields
DELETE	/candidates/:id	Remove a candidate
POST	/candidates/:id/onboarding/:itemId/toggle	Check off an onboarding checklist item

Record shape

```
{
  "id": "cand_331122", "title": "Senior Backend Engineer", "candidate": "Riya Mehta",
  "stage": "Applied",    // Applied | Screening | Interview | Offer | Hired
  "meta": "2d ago",
  "onboarding": [ { "id": "ob1", "label": "Verify documents", "done": false } ]
}
```

Roles

HR Manager, HR Director only.

Business logic to own

- Moving a candidate to Hired auto-generates a fixed onboarding checklist (today: 6 items).
- Resume upload does not exist as a real feature today - candidates show identical hardcoded fake resume text regardless of who they are, and 'download resume' just shows a toast. Building this for real means adding an actual file input to the frontend plus the file upload endpoint in Section 6, not just a backend change.
- The exit-interview / offboarding modal currently piggybacks on the onboarding checklist and writes free text into the audit log. Recommend a proper exit_interviews sub-resource instead of overloading onboarding once this is built for real.

5.7 Performance Reviews

Method	Path	Description
POST	/review-cycles	Body: cycleName. Bulk-creates one review row per employee
GET	/reviews	List review rows, optionally by cycle or employee
PATCH	/reviews/:id/self	Employee submits self rating/comments
PATCH	/reviews/:id/manager	Manager submits rating/comments
POST	/reviews/:id/goals	Add a goal
PATCH	/reviews/:id/goals/:goalId/toggle	Toggle a goal done/not done

Record shape

```
{
  "id": "rev_44a1", "cycleName": "H1 2026", "empId": "emp_a1b2c3", "name": "Ananya Nair",
  "dept": "Engineering", "status": "pending", // pending | self-submitted | completed
  "selfRating": null, "selfComments": "",
  "managerRating": null, "managerComments": "",
  "goals": [ { "id": "g1", "text": "Ship v2 API", "done": false } ]
}
```

Roles

Self section: the employee themselves. Manager section, and starting a cycle: HR Manager, HR Director.

Business logic to own

When a manager review is submitted, also update that employee's rating field on the Employees resource to the newly submitted managerRating - this is a real cross-resource side effect in the current frontend, not just a display value.

5.8 Expenses

Method	Path	Description
GET	/expenses	List expense claims
POST	/expenses	File a new claim
PATCH	/expenses/:id	Approve/decline with a reason, or edit

Record shape

```
{
  "id": "exp_7788", "empId": "emp_v9w8", "name": "Vikram Menon", "category": "Travel & Lodging",
  "amount": 4500, "date": "2026-07-01", "description": "Client meeting travel reimbursement",
  "receiptUrl": "", "status": "pending", "reason": ""
}
```

Roles

Create: Employee (self). Approve/decline: Finance Lead, HR Director.

Business logic to own

receiptUrl is currently a pasted text URL, not a real uploaded file (it defaults to a stock photo if left blank) - wiring in a genuine file upload uses the shared /files endpoint from Section 6 and needs a small frontend change to add a real file picker.

5.9 Settings

Method	Path	Description
GET	/settings	Get the org-wide settings object
PATCH	/settings	Update settings

Settings is a singleton, not a collection. Split it in the real backend: everything auth-sensitive (login accounts, passwords) moves out entirely into the real Users/Auth subsystem from Section 3 - it must never come back in a GET /settings response the way loginProfiles (including plaintext passwords) does in today's mock layer. Everything else - org name, work week, departments/designations, notification toggles, geofence configuration, notification templates, and gateway credential fields - can stay as one settings object.

Note: settings.approvalWorkflows is read and written by the frontend's Workflows page, but is never actually enforced by the real Leave/Expense approval logic anywhere in the app today. Keep the field for UI compatibility, but do not build server-side enforcement for it unless the product owner confirms multi-step approval is a real, near-term requirement - see Section 8.

6. File Upload and Storage Design

Three different 'file' touchpoints exist in the frontend today, at three different levels of reality. Documents.jsx does a genuine browser-side upload (FileReader to base64, capped at 2MB, mime-checked), while Recruitment resumes and Expense receipts are not real uploads at all yet (hardcoded text, and a pasted URL field, respectively). Design one shared subsystem that can serve all three once the missing frontend pickers are added.

6.1 Endpoints

Method	Path	Description
POST	/files	multipart/form-data: file, category. Returns id, url, mimeType, sizeBytes
GET	/files/:id	Streams or redirects to the stored file, respecting visibility rules
DELETE	/files/:id	Removes a stored file

Attach a `fileId` reference (not raw base64) to the owning record: `documents.fileId`, `expenses.receiptFileId`, `recruitment.resumeFileId`.

6.2 Storage

Start with local disk storage under an app-data volume - simplest option for a first self-hosted deployment. Design the storage layer behind a small adapter interface so it can be swapped for S3-compatible object storage (AWS S3, MinIO, etc.) later without changing the endpoint contract above.

6.3 Size limits and allowed types

Category	Max size	Allowed types
Documents (existing)	2 MB	PDF, DOC/DOCX, PNG/JPEG/WEBP, XLS/XLSX
Resumes (new)	Suggest 5 MB	PDF, DOC/DOCX
Receipts (new)	Suggest 5 MB	Image (PNG/JPEG) or PDF

Enforce these limits server-side - today they are only checked in the browser, so nothing stops a modified client from bypassing them.

6.4 Visibility

Documents already carry an `all / hr / finance` visibility tag. Carry this onto the file record so `GET /documents` can filter by the caller's role on the server - today the entire document list including HR-only items is sent to the browser regardless of role, and only hidden by a client-side filter, which is trivially bypassable.

6.5 Express example

```
// routes/files.js
const multer = require('multer');
const upload = multer({ limits: { fileSize: 5 * 1024 * 1024 } });

router.post('/', requireAuth, upload.single('file'), async (req, res) => {
  const allowed = CATEGORY_MIME_MAP[req.body.category] || [];
  if (!allowed.includes(req.file.mimetype)) {
    return res.status(400).json({ error: { code: 'BAD_FILE_TYPE', message: 'File type not allowed' } });
  }
  const stored = await storage.save(req.file, req.body.category);
  res.status(201).json({ id: stored.id, url: stored.url, mimeType: req.file.mimetype, sizeBytes: req.file.size });
});
```

7. Simulated Features That Need Real Implementation

A single reference table so nothing gets missed - each of these looks finished in the UI today but has no real implementation behind it.

Feature	What's fake today	What's needed
QR check-in	Cosmetic SVG QR code; scan auto-succeeds after a 3-second timer	Real signed, short-lived QR token - see Section 5.3
Biometric device sync	Device list and punches are randomly generated	Real device protocol integration (ZKTeco/eSSL SDK or gateway) - separate project, see Section 8
SMS / Email / WhatsApp sending	Templates and Twilio/SMTP fields exist; nothing is sent; UI itself labels this 'interface only for now'	Real provider integration (Twilio, SendGrid/SMTP) wired to the existing template fields
OTP / 2FA delivery	A fake code is shown to the user in an in-app toast	Real SMS or email delivery of the one-time code
GPS geofence check-in	Distance check only runs in the browser	Server must re-validate distance from submitted coordinates - never trust a client flag
Resume storage	Hardcoded identical fake text for every candidate; no file input exists	Real file upload (frontend + Section 6 endpoint) and per-candidate storage
Expense receipts	A pasted text URL, defaults to a stock photo if left blank	Real file upload via Section 6
Face descriptor persistence	Stored only in the browser's localStorage today	Optional: persist per-user descriptor server-side if you want server-checked biometric login

8. Non-Blocking and Future Items

These do not need to block a first backend release.

- **Approval Workflows** - the Workflows page saves a multi-stage approval configuration to settings, but no approval logic anywhere in the app actually reads or enforces it today. Build server-side enforcement only if the product owner confirms this is a genuine, near-term requirement.
- **Tally accounting export** - generated entirely client-side from data already in memory; it just needs to be fed real data once the API is live, no new backend endpoint is required.
- **Biometric hardware protocol integration** - talking to real ZKTeco/eSSL devices is a meaningfully larger, separate integration effort; today's Integrations page is a complete mock with no real device I/O.
- **Server-side face-recognition validation** - optional; the current client-side-only approach is a reasonable place to stay unless there's a specific security requirement to verify biometric login server-side.

Appendix: Full JSON Field Reference

Quick-lookup field lists for every entity, taken directly from the current frontend's seed data and data layer, without the surrounding prose above.

Employee

```
id, name, role, dept, loc, email, phone,  
status (active | remote | on-leave),  
joinDate, salary, rating, managerId, bankAccount, ifsc
```

Leave

```
id, empId, name, dept,  
type (sick | earned | casual),  
start, end, status (pending | approved | declined), reason
```

Attendance

```
id, empId, name, dept, date, checkIn, checkOut,  
status (present | late | absent | leave)
```

Payroll

```
id, empId, name, dept, gross, deductions, net,  
status (ready | processing | paid), cycle,  
lopDays, lopAmount, components: { earnings[], deductions[] }
```

Celebration

```
id, type (birthday | anniv), name, detail, wished
```

Holiday

```
id, name, date, type (National | Regional | Optional)
```

Candidate (Recruitment)

```
id, title, candidate,  
stage (Applied | Screening | Interview | Offer | Hired),  
meta, onboarding: [ { id, label, done } ]
```

Job Posting

```
id, title, department, location, type,  
status (Open | Closed), description
```

Performance Review

```
id, cycleName, empId, name, dept,  
status (pending | self-submitted | completed),  
selfRating, selfComments, managerRating, managerComments,  
goals: [ { id, text, done } ]
```

Expense

```
id, empId, name, category, amount, date, description,  
receiptUrl, status (pending | approved | declined), reason
```

Asset

```
id, name, category, serialNumber,  
status (assigned | available),  
assignedToEmpId, assignedToEmpName, assignedDate
```

Settings (singleton, auth fields removed - see Section 3)

```
orgName, workWeek, notifyLeave, notifyPayroll, notifyBirthday,  
totalLeaveDays, departments[], designations[],  
gpsCheckInEnabled, geofenceLat, geofenceLng, geofenceRadius,  
gatewayTwilioSid, gatewayTwilioToken, gatewayTwilioFrom,  
gatewaySendgridKey, gatewaySmtpHost, gatewaySmtpUser, gatewaySmtpPass,  
notificationTemplates: { email{}, sms{}, whatsapp{} },  
approvalWorkflows: { leave[], expense[] } // saved, currently unenforced
```